

# Functions

March 11, 2024

# main() - Calculator Program

```
#include <stdio.h>
#include <stdlib.h> /* for atof() */
#define MAXOP 100  /* max size of operand or operator */
#define NUMBER '0' /* signal that a number was found */

int getop(char []); void push(double); double pop(void);

void main() {          /* reverse Polish calculator */
    int type;
    double op2;
    char s[MAXOP];

    while ((type = getop(s)) != EOF) {
        switch (type) {
            case NUMBER:
                push(atof(s));
                break;
            case '+':
                push(pop() + pop());
                break;
```

# main() - Calculator Program

```
case '*':
    push(pop() * pop()); break;
case '-':
    op2 = pop();
    push(pop() - op2);
    break;
case '/':
    op2 = pop();
    if (op2 != 0.0) push(pop() / op2);
    else printf("error: zero divisor\n");
    break;
case '\\n':
    printf("\\t%.8g\\n", pop()); break;
default:
    printf("error: unknown command %s\\n", s); break;
}
}
return 0;
}
```

## push and pop - Calculator Program

```
#define MAXVAL 100 /* maximum depth of val stack */
int sp = 0;        /* next free stack position */
double val[MAXVAL]; /* value stack */

void push(double f) /* push f onto value stack */
{
    if (sp < MAXVAL)
        val[sp++] = f;
    else
        printf("error: stack full, can't push %g\n", f);
}

double pop(void) /* pop and return top value from stack */
{
    if (sp > 0)
        return val[--sp];
    else {
        printf("error: stack empty\n");
        return 0.0;
    }
}
```

# Function getop - Fetching Operators or Operands

```
#include <ctype.h>

int getch(void); void ungetch(int);

int getop(char s[]) {
    int i, c;
    while ((s[0] = c = getch()) == ' ' || c == '\t');
    s[1] = '\0';
    if (!isdigit(c) && c != '.') return c;
    i = 0;
    if (isdigit(c)) while (isdigit(s[++i] = c = getch()));
    if (c == '.') while (isdigit(s[++i] = c = getch()));
    s[i] = '\0';
    if (c != EOF) ungetch(c);
    return NUMBER;
}
```

# Input Handling Functions - getch and ungetch

```
#define BUFSIZE 100
char buf[BUFSIZE];
int bufp = 0;

int getch(void) {
    return (bufp > 0) ? buf[--bufp] : getchar();
}

void ungetch(int c) {
    if (bufp >= BUFSIZE)
        printf("ungetch: too many characters\n");
    else
        buf[bufp++] = c;
}
```

# Scope Rules in C

- ▶ Functions and external variables of a C program need not all be compiled at the same time
- ▶ Source text can be kept in several files; pre-compiled routines may be loaded from libraries.
- ▶ Questions of interest in C program organization:
  - ▶ How do we write declarations so as to ensure proper compilation?
  - ▶ How do we arrange declarations so that all are properly connected upon loading?
  - ▶ How are declarations organized so there is only one copy?
  - ▶ How are external variables initialized?
- ▶ Let us reorganize the calculator program into several files to illustrate these issues.

# Scope of a Name in C

- ▶ The scope of a name is the part of the program within which the name can be used.
- ▶ Automatic variables and parameters have scope from declaration to end of the function.
- ▶ Local variables and parameters are unrelated across functions.
- ▶ External variable or function scope lasts from declaration to the end of the file.
- ▶ If main, sp, val, push, and pop are defined in one file, in that order, then sp and val may be used in push and pop
- ▶ sp, val, push, pop are not visible in main.



# External Variable Declaration in C

- ▶ External variable or function needs extern declaration if used before definition in the same file.
- ▶ extern declaration is mandatory before use if definition is in a different source file.

```
extern int sp;  
extern double val[];  
    in file1.c
```

```
int sp = 0;  
double val[MAXVAL];  
    in file2.c
```

# Initialization of External Variables

- ▶ Initialization of external variables is done only with the definition.
- ▶ Array sizes must be specified with the definition. Storage is set aside at the definition.

## Example: Splitting Functions and Variables

- ▶ Functions and variables can be defined in separate files.
- ▶ Proper organization and declarations are necessary for connection.

% In file1:

```
extern int sp;  
extern double val[];
```

```
void push(double f) { ... }  
double pop(void) { ... }
```

% In file2:

```
int sp = 0;  
double val[MAXVAL];
```

# Header Files in C

- ▶ Dividing the calculator program into several source files for better organization.
- ▶ Components in different files: `main.c`, `stack.c`, `getop.c`, `getch.c`.
- ▶ Centralizing definitions and declarations using a header file: `calc.h`.
- ▶ Tradeoff between file access and maintenance for program organization.

# Program Structure - Files

- ▶ `main.c`: Contains the main function.
- ▶ `stack.c`: Contains `push`, `pop`, and related variables.
- ▶ `getop.c`: Contains the `getop` function.
- ▶ `getch.c`: Contains `getch` and `ungetch`.
- ▶ `calc.h`: Header file for common definitions and declarations.

## Header File - calc.h

```
#define NUMBER '0'  
void push(double f);  
double pop(void);  
int getop(char s[]);  
int getch(void);  
void ungetch(int c);
```

## main.c

```
#include <stdio.h>
#include <stdlib.h>
#include "calc.h"
#define MAXOP 100
int main() {
    // Main function code
    return 0;
}
```

## stack.c

```
#include <stdio.h>
#include "calc.h"
#define MAXVAL 100
int sp = 0;
double val[MAXVAL];

void push(double f) {
    // push implementation
}

double pop(void) {
    // pop implementation
}
```



## getop.c

```
#include <stdio.h>
#include <ctype.h>
#include "calc.h"

int getop(char s[]) {
    // getop implementation
}
```

## getch.c

```
#include <stdio.h>
#include "calc.h"
#define BUFSIZE 100
char buf[BUFSIZE];
int bufp=0;
int getch(void) {
    // getch implementation
}

void ungetch(int c) {
    // ungetch implementation
}
```

# Selection Sort

- ▶ Say, the elements to sort occupy positions 0 to  $n - 1$  of an array.
- ▶ for  $i = 0$  to  $n - 2$ 
  - ▶ /\* The elements in positions upto  $i - 1$  are sorted. \*/
  - ▶ Find the smallest among the elements in positions  $i$  to  $n - 1$ .
  - ▶ Swap it with the  $i$ -th element.

Time taken =  $n + (n - 1) + \dots + 2 = O(n^2)$ .

# Selection Sort - Example Walkthrough

**Initial Array:** [64, 25, 12, 22, 11]

**Iteration 1:** Select 11 (min) and swap with the 1st element.

**Array:** [11, 25, 12, 22, 64]

**Iteration 2:** Select 12 (min) and swap with the 2nd element.

**Array:** [11, 12, 25, 22, 64]

**Iteration 3:** Select 22 (min) and swap with the 3rd element.

**Array:** [11, 12, 22, 25, 64]

**Iteration 4:** Select 25 (min) and swap with the 4th element.

**Sorted Array:** [11, 12, 22, 25, 64]